CONTENT-BASED IMAGE RETRIEVAL WITH SCALE INVARIANT FEATURE TRANSFORM (SIFT) AND PARTIAL PROCESSING DESIGN

A Project

Presented to the

Faculty of

California State Polytechnic University, Pomona

In Partial Fulfillment

Of the Requirements for the Degree

Master of Science

In

Computer Science

By

Sairaj Jadhav

2023

SIGNATURE PAGE

PROJECT:	CONTENT-BASED IMAGE RETRIEVAL WITH SCALE INVARIANT FEATURE TRANSFORM (SIFT) AND PARTIAL PROCESSING DESIGN
AUTHOR:	SAIRAJ JADHAV
DATE SUBMITTED:	Spring 2023
	Department of Computer Science

Dr. John Korah
Project Committee Chair _____
Department of Computer Science

Dr. Mohammad Husain
Professor _____
Department of Computer Science

ACKNOWLEDGMENT

I would like to express my sincere gratitude and appreciation to my project advisor, Dr. John Korah, for their invaluable guidance, support, and expertise throughout the duration of my project. Their insightful feedback, constructive criticism, and continuous encouragement have been instrumental in shaping the direction and success of this project. I am truly grateful for their unwavering commitment, patience, and dedication in providing me with the necessary resources and mentorship.

I would also like to extend my gratitude to my committee member, Dr. Mohammad Husain, for their valuable input and contributions to this project. Their expertise, suggestions, and critical evaluation have significantly enriched the quality and depth of my research. I am thankful for their time and effort in reviewing my work, providing valuable insights, and helping me refine my ideas.

Additionally, I would like to acknowledge the collective support and encouragement I have received from the entire committee. Their involvement and guidance have played a crucial role in shaping the overall development and outcome of this project.

Lastly, I would like to express my appreciation to the institution, department their resources, facilities, and encouragement have been invaluable in making this research endeavor possible. Once again, I extend my heartfelt thanks to my project advisor and committee member for their unwavering support, expertise, and guidance throughout this project. Their contributions have been pivotal in my academic and personal growth, and I am grateful for the opportunity to have worked with them.

ABSTRACT

The aim of this technical report is to enhance the computational efficiency of the Scale Invariant Feature Transform (SIFT) algorithm, which is widely used for content-based image retrieval. The conventional sequential approach in SIFT poses challenges in terms of computational resources and processing time. To address these challenges, we explore methodologies that can reduce the computation required for keypoints while retaining essential features. One approach is the integration of Non-maximum Suppression (NMS) in the SIFT algorithm, which identifies and removes redundant keypoints. The NMS-SIFT method demonstrates a better matching rate and faster processing time compared to the traditional SIFT. Moreover, to further optimize the computation time, we implement a multithreaded approach where the image is divided into quadrants processed independently by separate threads. This technique significantly improves processing time while accurately mapping keypoints to their corresponding quadrants. Additionally, we investigate partial processing with quadrant-based image division, which allows for efficient processing of large images by reducing the computational burden. Experimental results demonstrate the effectiveness of partial processing in extracting features while optimizing processing time. The insights gained from quadrant processing can be utilized to determine whether to continue or stop processing an image based on the desired number of features. Overall, the methodologies presented in this report contribute to improving the computational efficiency of the SIFT algorithm and have implications for real-time image processing and retrieval tasks.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF EQUATIONS

CHAPTER 1: INTRODUCTION

With the drastic rise in available digital image data, there has been an increase in the need for more robust and efficient image processing. Content-Based Image Retrieval (CBIR) (Zhou, Li, and Tian 2017) (Zhang 2002) attempts to address the increased demand by providing a consistent, automated way to compare and retrieve images. Content-Based Image Retrieval (CBIR) is a technique that helps find and retrieve images from a database based on their visual content. Instead of relying on labels or text descriptions, CBIR systems analyze the actual features of the images, such as colors, textures, shapes, and relationships between objects. These features are used to create a unique representation for each image in the database. When a user searches for an image, the system compares the visual features of the query image to those in the database and ranks them based on similarity. This allows users to find images that look similar or have similar visual characteristics. CBIR is useful in various areas that manage large image databases, medical imaging, surveillance, and more, as it provides an efficient way to retrieve images based on their visual content.

Features in the image are nothing but important pieces of information that help identify and understand an image. Feature extraction is the process of finding and retrieving these important details from an image. These details are crucial for tasks like image classification and object recognition. The goal of feature extraction is to gather a set of features that accurately represent the content of the image and help in analyzing it further.

CBIR (Content-Based Image Retrieval) poses a disadvantage due to its heavy computational resource requirements. One of the contributing factors is the large amount

of image data involved. As the image data increases, the computational demands also escalate. Additionally, real-time applications impose further pressure on the available computational resources, as CBIR needs to operate in real-time. CBIR algorithms can be intricate, involving multiple steps such as image pre-processing, feature extraction, and matching, all of which demand significant computational capabilities to accomplish.

In this research, we focus on a well-known CBIR algorithm- Scale Invariant Feature Transform (SIFT) (Lowe 2004), and we experiment with diverse ways to expedite the process to make it more applicable to real-time image processing and retrieval. The Scale Invariant Feature Transform (SIFT) algorithm offers several unique advantages that set it apart from other CBIR (Content-Based Image Retrieval) algorithms. One notable characteristic is its focus on locality. SIFT features are extracted from small patches or regions within an image, making them resilient to clutter and occlusion. Another distinguishing feature is the distinctiveness of SIFT features. They possess the ability to be matched with a large dataset of objects, even in the presence of variations in scale, rotation, and illumination. Additionally, SIFT generates a substantial number of features, even for small objects, making it highly effective for image matching tasks. These distinctive qualities make the SIFT algorithm a valuable tool for robust and accurate content-based image retrieval.

In comparison to other CBIR algorithms, the Scale Invariant Feature Transform (SIFT) algorithm offers several advantages. Firstly, SIFT generates a greater number of key points, providing a richer representation of image features. It also exhibits robustness to various image transformations, making it suitable for handling diverse and complex images. The matching rate of SIFT is generally superior, contributing to more accurate

and reliable image retrieval. Additionally, SIFT is a well-established algorithm that has been widely adopted in the computer vision community. Implementing the Scale-Invariant Feature Transform (SIFT) algorithm comes with its share of challenges. One of the major hurdles is the computational overhead it entails. SIFT comprises a series of interconnected steps, where the output of one stage serves as the input for the next. Processing all these steps simultaneously can be quite challenging.

Real-time image processing is another obstacle when it comes to implementing SIFT. The algorithm involves multiple computationally intensive steps, making it difficult to achieve real-time performance. Each stage, such as scale-space extrema detection, keypoint localization, orientation assignment, and descriptor generation, requires significant computational resources. Memory usage is also a concern when implementing SIFT. The algorithm demands substantial memory to store intermediate results and descriptors. As the number of keypoints and the complexity of the images increase, the memory requirements also grow, posing limitations on systems with limited memory capacity. Addressing these challenges is crucial to effectively implement the SIFT algorithm and unlock its potential for various applications. Optimizations in computational efficiency, memory management, and real-time processing are necessary to overcome these hurdles and make SIFT more accessible and practical in real-world scenarios.

CHAPTER 2 : LITERATURE SURVEY

In this section, we provide a discussion of content-based image retrieval (CBIR) techniques as precursor to understanding our methodology and state of the art in the domain. Content-based image retrieval (Zhou, Li, and Tian 2017) aims to meet the growing need for a reliable and automated method to compare and retrieve images. It accomplishes this by comparing the visual features and characteristics of images to retrieve similar ones. CBIR finds applications in diverse fields, including medical diagnosis, object detection, fingerprint analysis, and forensic science. Additionally, we will examine various SIFT architectures that have been implemented in the past.

In CBIR, the user specifies a query image and retrieves images in the database similar to the query image. To find the most similar images, CBIR compares the content of the input image with database images. More specifically, CBIR compares visual features such as shape, color, texture, and spatial information to measure the similarity between a query image and images in a database related to those features. Content-based image retrieval (CBIR) is a better technique compared to text-based image retrieval for retrieving images. In a Text-Based Image Retrieval (TBIR) (baeldung 2022) approach as the search query depends on user-defined meta tags there is a chance of human error in retrieval.

Content-based image retrieval (CBIR) algorithms are known for their complexity, involving various processing steps to achieve accurate image retrieval. These steps typically include image pre-processing, feature extraction, and matching. While CBIR can yield impressive results, it comes with the drawback of demanding significant

computational resources. The computational requirements pose a challenge and can limit the real-time applicability of CBIR systems. Figure 1 provides a visual representation of the intricate architecture involved in CBIR algorithms, highlighting the complexity of the process.

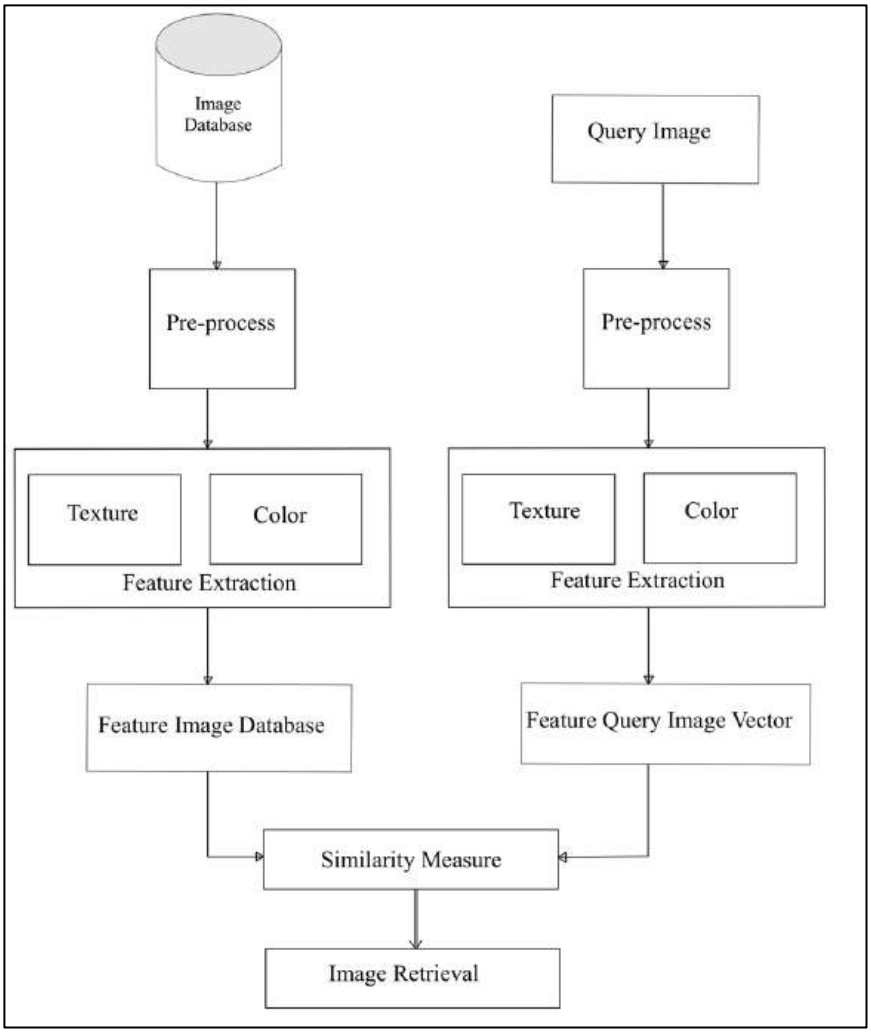

Figure 1: CBIR (adapted from Jayaswal and Jha 2017)

Feature extraction (Kabbai, Abdellaoui, and Douik 2018) is one of the most important stages in any CBIR algorithm. Feature extraction plays a crucial role in computer vision tasks by identifying and extracting significant visual information from an image. These

extracted features serve as the foundation for subsequent analysis, such as image classification or object recognition. The primary objective of feature extraction is to capture and represent the essential characteristics of an image in a compact and informative manner. By extracting discriminative features, the subsequent analysis algorithms can make accurate decisions based on the extracted information. Effective feature extraction techniques enable the extraction of relevant information while minimizing noise and irrelevant details, leading to improved performance in various computer vision applications.

The features in the image are described into two types:

1. Global Features
2. Local Features

2.1 Global Features

Global features (Das 2015) (Wang, Wu, and Yang 2009) (Haralick, Shanmugam, and Dinstein 1973) (Stylianou and Farin 2003) are those that describe an entire image. They contain information on the entire image. Global feature-based image retrieval algorithm uses a single feature vector to represent the entire image. No attention is paid to the constituents of the image, such as individual regions or objects. For image retrieval, there are many different global features which can be used. Some of the global features are:

- Color Histogram (Wang, Wu, and Yang 2009): The distribution of color values in an image is represented by a color histogram, which can be used to describe the color content of the image.
- Texture features (Haralick, Shanmugam, and Dinstein 1973): These features explain how edges, lines, and other texture elements are arranged spatially in an image.

- Shape features (Stylianou and Farin 2003): The geometric features of objects in an image, such as size, orientation, or curvature, are captured by these features.

  Some of the algorithm which make use of global features are:

  a. Color Histogram (Wang, Wu, and Yang 2009): This algorithm extracts the color distribution of an image and presents it as a histogram. Similarity between two images can be calculated using Euclidean distance.
  b. Global Image Descriptor (GIST) (Kabbai, Abdellaoui, and Douik 2018) This algorithm extracts the global spatial arrangement of an image and represents it as a feature vector. The similarity between two images can be calculated using Euclidean distance or cosine similarity.
  c. Dense SIFT (Liu and Wang 2015): This algorithm is an extension of SIFT that densely separates objects from a grid of points. It constructs a global feature vector by connecting local graphs. The similarity between two images can be calculated using Euclidean distance or cosine similarity.
  d. Convolutional Neural Network(CNN) (Alzubaidi et al. 2021): This deep neural network-based modeling technique learns visual features using a hierarchical representation. The similarity between two images can be calculated using cosine similarity or other distance measures of feature vectors extracted from the network.

2.2 Local Features

Local features (Mikolajczyk and Tuytelaars 2009) (Lowe 2004) (Bay et al. 2008) (Rublee et al. 2011) (Leutenegger, Chli, and Siegwart 2011) (Alexandre Alahi, Ortiz, and Vandergheynst 2012)While global features have many advantages, they change under

scaling and rotation. For this reason, local features are more reliable in various conditions. Local features describe visual patterns or structures identifiable in small groups of pixels. Examples of local features include edges, points, and various image patches.

The descriptors used to extract local features consider the regions centered around the detected visual structures. Those descriptors transform a local pixel neighborhood into a vector presentation. Following are few of the local feature image retrieval algorithms:

- Scale-Invariant Feature Transform (SIFT) (Lowe 2004): This algorithm detects and extracts scale-invariant keypoints from an image and computes a descriptor for each keypoint. The similarity between two images can be calculated using the Euclidean distance or the cosine similarity of the descriptors.
- Speeded Up Robust Features (SURF) (Bay et al. 2008): This algorithm is similar to SIFT but is designed to be faster and more robust. It also detects and extracts scale-invariant keypoints and computes a descriptor for each keypoint. The similarity between two images can be calculated using the Euclidean distance or the cosine similarity of the descriptors.
- Oriented FAST and Rotated BRIEF (ORB) (Rublee et al. 2011): This algorithm combines the speed of the FAST detector and the robustness of the BRIEF descriptor to detect and describe keypoints in an image. The similarity between two images can be calculated using the Hamming distance of the descriptors.
- Binary Robust Invariant Scalable Keypoints (BRISK) (Leutenegger, Chli, and Siegwart 2011): This algorithm is similar to ORB but uses a scale-space pyramid to detect keypoints at different scales. It also uses a more complex descriptor that

is robust to geometric and photometric transformations. The similarity between two images can be calculated using the Hamming distance of the descriptors.

- Fast Retina Keypoint (FREAK) (Alexandre Alahi, Ortiz, and Vandergheynst 2012): This algorithm uses a binary descriptor that is inspired by the human retina to describe keypoints in an image. The similarity between two images can be calculated using the Hamming distance of the descriptors.
- Accelerated-KAZE (AKAZE) (Alcantarilla 2012): This algorithm is based on the KAZE algorithm and is designed to be faster and more efficient. It uses a nonlinear scale space to detect keypoints and computes a descriptor for each keypoint using the M-SURF descriptor. The similarity between two images can be calculated using the Euclidean distance or the cosine similarity of the descriptors. (Kashif et al. 2016)

These techniques have been demonstrated to provide excellent accuracy in local feature matching and image retrieval and are widely used in computer vision applications.

Oriented FAST and Rotated BRIEF (Rublee et al. 2011) and Speeded-Up Robust Features (Bay et al. 2008) are two widely used CBIR algorithms. As noted above, ORB is a fusion of the FAST key point detector and BRIEF descriptor with some modifications. SURF approximates the DoG with box filters. Instead of Gaussian averaging the image, squares are used for approximation since the convolution with squares is much faster if the integral image is used. Karami *et al* (Karami, Prasad, and Shehata 2017) compared the results of the performances of these three algorithms i.e., SIFT, SURF, and ORB. The research provides the number of key points generated, the time is taken, and the matching

rate. The simulated results were used to investigate the sensitivity of SIFT, SURF and ORB against intensity, rotation, scaling, shearing, and noise.

The results obtained proved that the SIFT algorithm has the highest matching rate compared to the other algorithms and has an almost equivalent number of key points detected as the ORB. The only issue with the SIFT is the amount of time taken to generate the key points. It is higher than both other algorithms. It suggests that tackling the computational fragment of the SIFT will render it more feasible in application domains.

The research by Kumar et al (Kumar and Bhatia 2014) discusses GPU parallelization strategies for Scale-Invariant Feature Transform (SIFT). The proposed strategies in the paper are divided into two phases, the algorithm design phase, and the implementation design phase. The algorithm design phase focuses on parallelizing SIFT based on data size, usage, and organization and utilizes three guidelines: Reduce, Reuse, and Re-arrange. The Reduce guideline involves reducing memory operations to increase the computational intensity (CI) of the algorithm, Reuse involves maximizing the reusability of memory elements, and rearrange involves rearranging data to avoid branching and increase memory bandwidth usage. The implementation design phase focuses on maximizing the execution efficiency of the parallelized SIFT on a GPU. The research mentions that the Sub-Scale Extrema Detection (SSED) parallelization has been improved by adhering to the guidelines in the algorithm design phase. The research also mentions the problem with the pyramid structure of SSED in terms of performance deterioration for high-resolution images and suggests a top-down approach for

parallelization that groups data based on the top Difference of Gaussians (DoG) image in the highest octave. (Li *et al*. 2013)

In domains such as remote sensing, traffic monitoring requires real-time high-resolution image processing. SIFT has proven good capability in all such domains. But since the SIFT algorithm has such a huge computational burden, the use of SIFT in such applications is not feasible. Previously, many researchers have attempted to parallelize SIFT to make it applicable in real-time. The research by Li *et al* (Li 2013) demonstrated the potential for parallelizing SIFT using their multi-core parallelization design. But the research suggested the need for future work involving performance for larger images which is a challenging task. In another research by Li *et al* (Li et al. 2013) propose a distributed parallel algorithm (DDP-SIFT) to accelerate the process of extracting Scale Invariant Feature Transform (SIFT) features in a distributed environment. The proposed algorithm uses a special data parallel approach that divides the Gauss Scale Space into octaves to acquire large image blocks. To make this approach effective, the steps of building Gauss Scale Space were changed, and only a prerequisite part was produced before task allocation, reducing the Allocation Data Quantity (ADQ) by 13 times. The DDP-SIFT algorithm also proposed a refined-blocking approach to improve load balance. The experimental results showed that the proposed algorithm significantly improved the performance of SIFT features extraction while pursuing large data blocks.

The SIFT algorithm has gained widespread acceptance in the computer vision community, establishing itself as the standard for feature extraction and matching. Its implementations can be found in numerous research papers, libraries, and software tools, ensuring its accessibility and compatibility across different systems. The versatility of the

SIFT algorithm is evident in its broad range of applications, including object recognition, image stitching, 3D reconstruction, augmented reality, and content-based image retrieval. Its effectiveness and adaptability in various domains contribute to its popularity and extensive usage. Furthermore, the SIFT algorithm has undergone thorough evaluation through extensive studies and benchmarks, demonstrating its robustness and efficacy in numerous experiments and challenges. It has been rigorously tested on diverse datasets and under different conditions, solidifying its reputation for reliability and utility. Therefore, we have selected the SIFT algorithm as the basis for our implementation of a more efficient content-based image retrieval system.

CHAPTER 3: RESEARCH PROBLEM

Our main objective is to improve the computational efficiency of the SIFT algorithm. The conventional sequential approach in SIFT computes all the keypoints, which poses a challenge in terms of computational resources. Furthermore, each identified keypoint requires additional processing to generate descriptors, which involve extensive computations. To enhance the efficiency of the SIFT algorithm, we need to explore methods that can reduce the computation required for keypoints while retaining crucial features. One approach is to identify keypoints that describe similar features, even if they are not identical. By doing so, we can identify redundant features and potentially reduce the overall computational load without compromising essential information.

3.1 Research Challenges

This chapter aims to address and discuss the various challenges faced throughout the research process, shedding light on the hurdles that needed to be overcome to achieve the desired objectives.

3.1.1 Computational Overhead:

The SIFT algorithm presents a computational challenge due to its multi-step nature and sequential processing flow. Each step in the algorithm relies on the output of the previous stage, creating a chain of interdependence. Executing all the steps simultaneously or in parallel becomes a complex task, as the output of one stage serves as the input for the subsequent stage. This interdependence and sequential nature of the SIFT algorithm makes it difficult to achieve simultaneous or parallel processing,

requiring careful consideration and implementation strategies to optimize its computational efficiency.

3.1.2 Real-Time Image Processing:

Implementing the SIFT algorithm for real-time image processing poses several challenges due to its computationally expensive nature. Each step involved in the algorithm, including scale-space extrema detection, keypoint localization, orientation assignment, and descriptor generation, contributes to the overall processing time. The cumulative effect of these steps can result in significant delays, making it difficult to achieve real-time performance in certain scenarios. The computational overhead of the SIFT algorithm requires careful consideration and optimization strategies to ensure efficient and timely image processing in real-time applications.

3.1.3 Memory Usage:

The memory usage of the SIFT algorithm is a significant consideration in its implementation. Storing intermediate results and descriptors in memory can consume a substantial number of resources. The memory requirements scale with the number of keypoints detected and the complexity of the images being processed. In scenarios involving large-scale image datasets or resource-constrained environments, the high memory usage of the SIFT algorithm can become a limiting factor. Efficient memory management techniques and optimization strategies are crucial to ensure the algorithm's feasibility and practicality in such situations. Addressing the memory usage challenge is essential for maximizing the algorithm's efficiency and broadening its applicability in various real-world scenarios.

3.2 Research Objectives

This research aims to experiment with different ways to expedite the SIFT algorithm to make it more applicable for real-time image processing and retrieval. We aim to implement various optimization techniques to improve the algorithm's efficiency without compromising the accuracy of feature extraction. We have the following specific research objectives.

- One of the main techniques used is the Non-Maximum Suppression. The NMS - SIFT algorithm provides faster processing times than the traditional algorithm. (Bodla et al. 2017) As we investigate components that allow us to decrease the computation of keypoints while maintaining crucial features we detect keypoints that describe comparable features. The goal is to employ the NMS threshold technique to identify keypoints that could have redundant information or are within the overlapping threshold.
- To further improve processing speed, we propose a parallel approach that employs multithreading to process the image in quadrants, rather than processing the whole image at once.
- We analyze the results obtained from each quadrant to determine whether to continue or terminate processing for the corresponding image.

This section addressed the research problem, research objectives, and research challenges of the project. The next section is the Technical Background, which will provide a comprehensive overview of the fundamental concepts and techniques related to the project, laying the foundation for the subsequent discussions and methodologies.

CHAPTER 4 : TECHNICAL BACKGROUND

The practical implementation of the Scale-Invariant Feature Transform (SIFT) algorithm can be costly due to the need for specialized hardware architecture. To address this challenge, we propose the utilization of optimization techniques aimed at enhancing the efficiency of the algorithm while preserving the accuracy of feature extraction. One of the key approaches adopted in this project is the implementation of Non-Maximum Suppression (NMS) (Bodla et al. 2017). NMS is a widely used computer vision technique that aims to eliminate redundant or overlapping keypoints from the feature set. By applying NMS, we can reduce computational overhead and streamline the processing pipeline by discarding keypoints that do not require further analysis. This optimization step contributes to improving the overall efficiency and performance of the SIFT algorithm implementation.

To lay the foundation for this project's implementation, we provide an overview of the fundamental techniques that play a key role.

4.1 Scale Invariant Feature Transform

The Scale Invariant Feature Transform (SIFT) is one of the popular CBIR methodologies for extracting meaningful information from images. SIFT is an image retrieval algorithm that has been optimized. The SIFT algorithm takes an image as input and produces vectors that represent the image's features as output. It extracts invariant features that are distinctive. It means that the retrieved features are scale, rotation, and viewpoint invariant. The problem of Image Matching is addressed by SIFT. Many issues in computer vision, such as object or scene recognition, solving for 3D structure from

multiple images, stereo correspondence, and motion tracking, involve image matching. They are well-localized in both the spatial and frequency domains, lowering the likelihood of occlusion, clutter, or noise causing disturbance. With effective algorithms, many features can be retrieved from common images. Furthermore, the features are quite different, allowing a single feature to be accurately matched with a high probability against a massive database of features, laying the groundwork for object and scene identification.

This method generates a high number of features that densely cover the image at all scales and locations, which is an important property. A typical image of 500x500 pixels yields approximately 2000 stable features (Lowe 2004) (although this number depends on both image content and choices for various parameters). The number of features is especially significant in object recognition, where the capacity to recognize small items in cluttered backgrounds necessitates the proper matching of at least three attributes from each object for reliable identification.

4.1.2 The Stages in SIFT

The SIFT (Scale-Invariant Feature Transform) algorithm comprises a series of cascading steps, where the output of one stage serves as the input for the next stage. This sequential flow of information allows for the extraction and refinement of key features throughout the process. However, performing all the steps simultaneously can be challenging due to the complexity and interdependence of the computations involved. Therefore, it is necessary to follow the step-by-step approach to ensure accurate feature extraction and achieve reliable results.

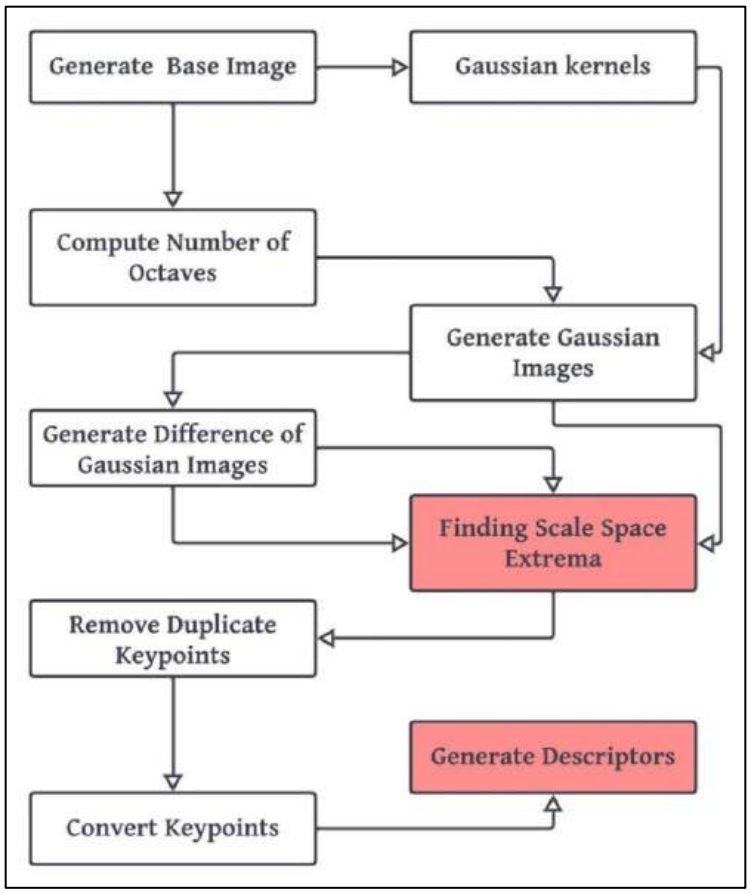

Figure 2: Stages in SIFT (Lowe 2004)

Figure 2 illustrates the interdependence of the various stages within the SIFT algorithm. It visually presents the sequential flow and interconnectedness of these stages, highlighting how each step builds upon the output of the previous step. This interdependency ensures the integrity and accuracy of the overall SIFT process, as the information extracted in one stage serves as a crucial input for subsequent stages. The figure provides a clear visual representation of the cohesive nature of the algorithm and how each stage contributes to the result. The stages which have high computational complexity are highlighted in red.

The key computational stages needed to generate the set of image features are as follows:

a) Scale-space extrema detection:

The initial stage of computation in the SIFT algorithm involves searching for keypoints across different scales and image locations. This process is efficiently implemented using a difference-of-Gaussian function, which helps identify potential interest points that exhibit scale and orientation invariance. To detect these keypoints, a cascading filter approach is employed. This approach utilizes efficient algorithms to identify candidate locations that are further examined in detail. By applying this cascading filter, the algorithm narrows down the search space, focusing on areas with a higher likelihood of containing informative keypoints. The scales space of an input image, I (x, y) image is defined as a function, L (x, y, σ), that is produced from the convolution of a variable-scale Gaussian, G(x, y, σ), where, x and y are coordinates of pixel and σ is the gaussian blur coefficient, as represented below.

$$L\ (x, y, \sigma) = G\ (x, y, \sigma) * I\ (x, y)$$

Equation 1: Generating Laplacian of Gaussian

$*$ is the convolution operation in x and y, and

$$G\ (x, y, \sigma) = (1 / 2\Pi\ \sigma^{2)}\ e^{-(x2\ +\ y2)}\ /\ 2\ \sigma^2$$

Equation 2: Gaussian filter

Figure 3: Base Image

Figure 3 serves as the base image utilized to generate the image pyramid for subsequent processing.

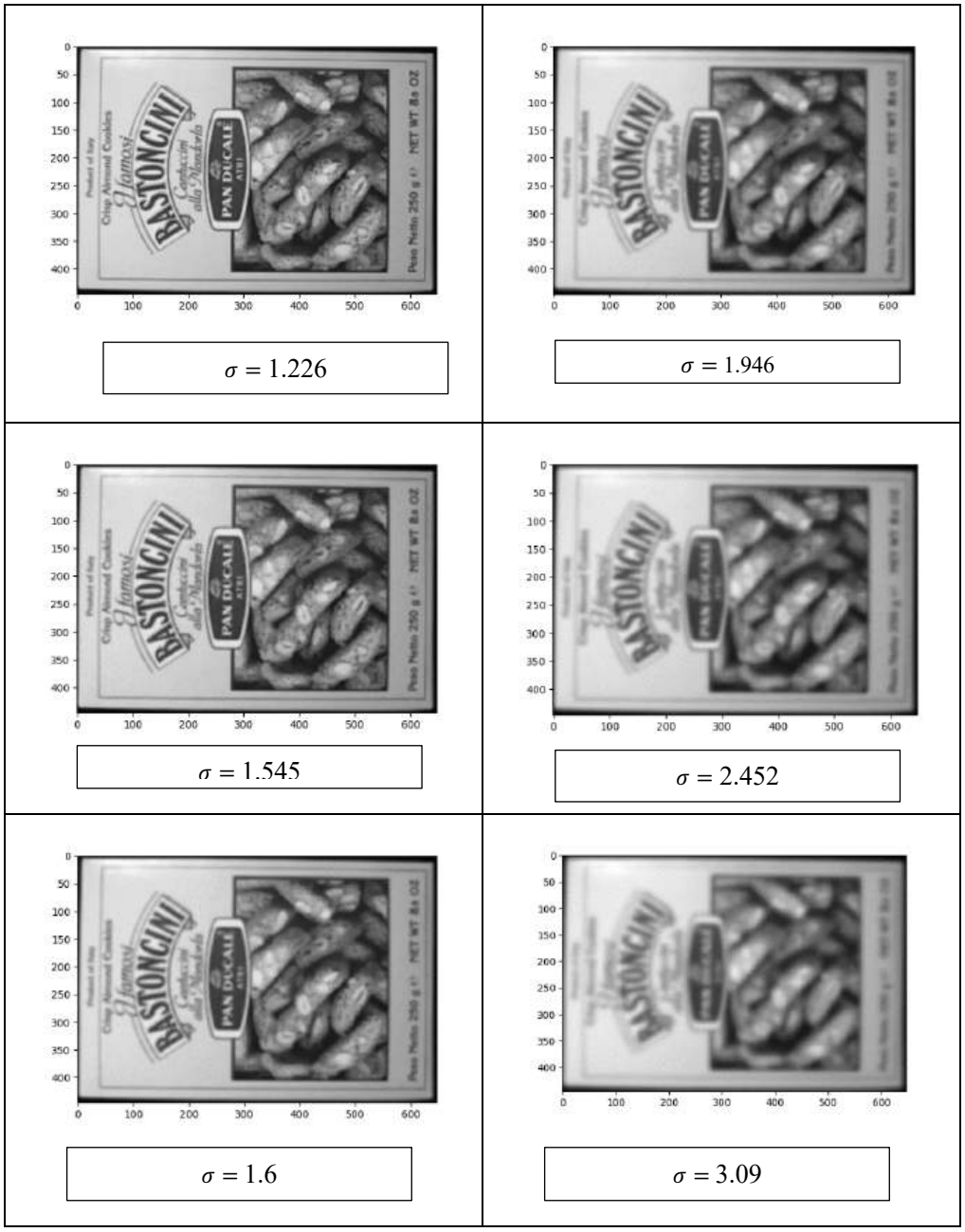

Figure 4: First Octave Laplacian of Gaussian of Base Image

Figure 4 illustrates the first octave of the Laplacian of Gaussians, which demonstrates a sequence of images with increasing blurring as we descend through the octave. In each level of the octave, the blurring coefficient, sigma, is progressively incremented, resulting in images that become progressively more blurred.

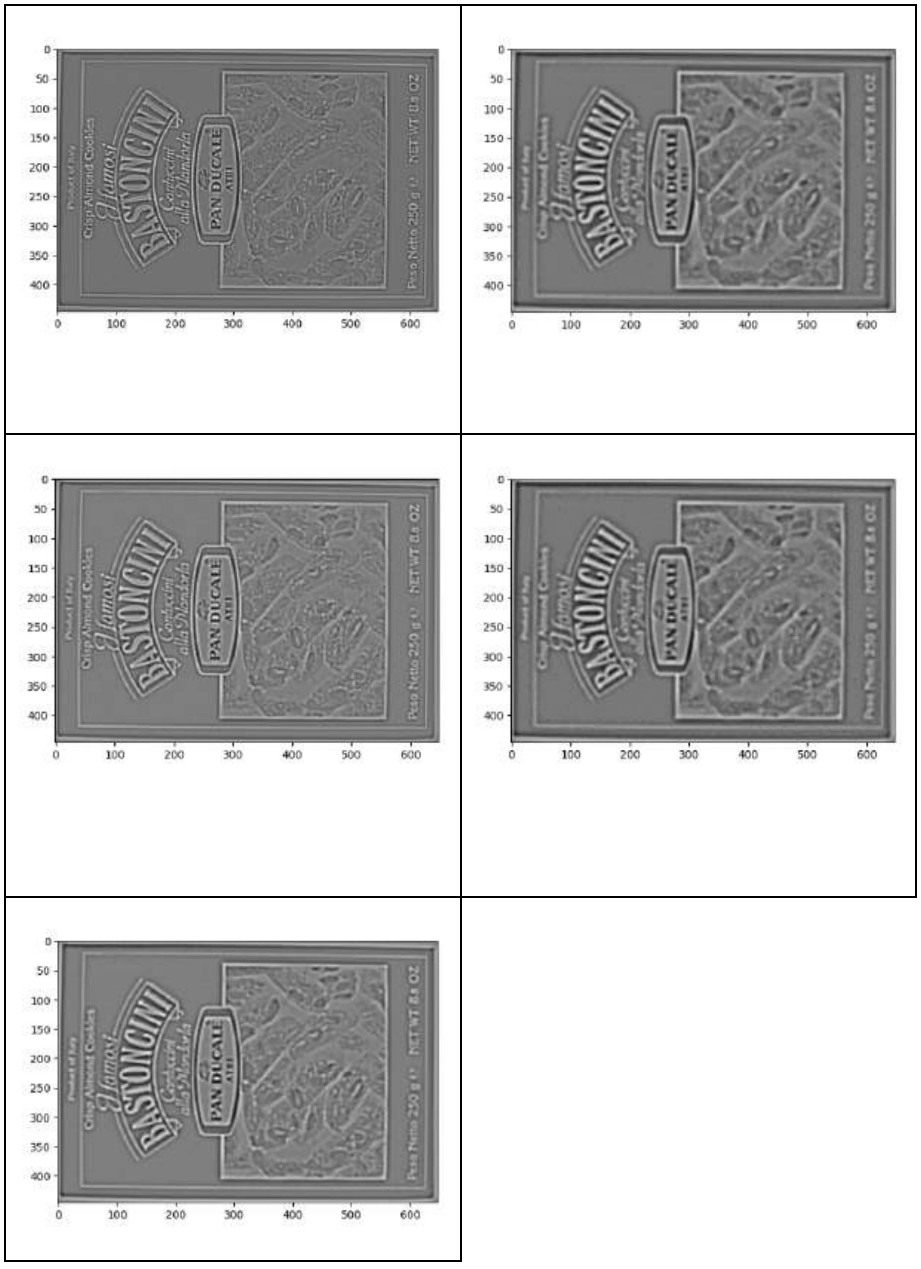

Figure 5: First Octave of Difference of Gaussians of the Base Image

Figure 5 illustrates the first octave of the difference of gaussian pyramid. To efficiently detect stable keypoint locations in scale space, scale-space extrema in the difference-of-Gaussian function convolved with the image, D (x, y, σ), which can be computed from the difference of two nearby scales separated by a constant multiplicative factor k:

$$D(x, y, \sigma) = \big(G(x, y, k\sigma) - G(x, y, \sigma)\big) * I\,(x, y)$$

$$L(x, y, k\sigma) - L(x, y, \sigma)$$

Equation 2: Computing Difference of Gaussian

There are several reasons for choosing this function. First, it is a particularly efficient function to compute, as the smoothed images, L, need to be computed in any case for scale space feature description, and D can therefore be computed by simple image subtraction. For each octave of scale space, the initial image is repeatedly convolved with Gaussians to produce the set of scale space images shown on the left. Adjacent Gaussian images are subtracted to produce the difference-of-Gaussian images on the right.

After each octave, the Gaussian image is down sampled by a factor of 2, and the process repeated. The maxima and Minima of the difference-of-Gaussian images are detected by comparing a pixel to its 26 neighbors (8 in same scale and 9 in the adjacent scales) in 3x3 regions at the current and adjacent scales (marked with circles)

b) Keypoint localization

During the keypoint selection process, certain keypoints are eliminated based on their characteristics. Keypoints with low contrast or those lying on edges are among the keypoints that are discarded. To identify low contrast keypoints, the intensities of the keypoints are examined. If the intensity at the detected extrema falls below a specified

threshold value (0.03 by Lowe *et al* in 2004), the keypoint is rejected. In addition to eliminating low contrast keypoints, keypoints located on edges are also rejected. This is achieved by computing the Hessian Matrix (H), which is a 2x2 matrix containing second-order partial derivatives of the intensity values with respect to the spatial coordinates. By analyzing the Hessian Matrix, the principal curvatures can be determined. The principal curvatures represent the curvatures along the two orthogonal directions at a given keypoint. If the principal curvature exceeds a predefined threshold (denoted as r), indicating that the keypoint lies on an edge, it is rejected. By applying these criteria for elimination, the algorithm ensures that only keypoints with sufficient contrast and that are not located on edges are considered for further processing and feature extraction.

c) Orientation assignment

To achieve rotation invariance, the SIFT algorithm calculates the orientation and gradient magnitude for each keypoint. This step is crucial for ensuring that the features extracted from the keypoints are not affected by the image's rotation. For each keypoint, an orientation histogram is generated, consisting of 36 bins covering a full 360-degree range. The purpose of this histogram is to capture the distribution of gradient orientations in the neighborhood of the keypoint. Peaks in the histogram indicate the dominant directions of gradients around the keypoint.

$$m(x,y) = \sqrt{(L(x+1,y) - L(x-1),y))2 + (L(x,y+1)} - L(x,y-1))2$$

Equation 3: Gradient magnitude

$$\theta(x,y) = tan^{-1}\frac{L(x,y+1) - L(x,y-1)}{L(x+1,y) - L(x-1,y)}$$

Equation 4: Gradient orientation

The calculation of the orientation and gradient magnitude relies on two parameters, $m(x,y)$ and $\theta(x,y)$.The parameter $m(x,y)$ represents the strength or magnitude of the change in pixel intensity at a given point (x,y)in the image. On the other hand, $\theta(x,y)$represents the direction or angle of the gradient at that particular point. By analyzing the orientation histogram and identifying the dominant directions, the SIFT algorithm achieves robustness against image rotation. This rotation invariance property enhances the algorithm's ability to match and recognize keypoints across images with varying orientations.

d) **Keypoint descriptor**

The keypoint descriptor extracts a distinctive representation of each keypoint by considering the gradient magnitude and orientation within subregions of the neighborhood of the keypoint. This information is aggregated using orientation histograms, which highlight the dominant orientations. To ensure rotation invariance, the descriptor is aligned based on the dominant orientation, with the histogram bins

reweighted accordingly. The final descriptor is a vector that concatenates histogram values from all subregions, capturing the unique local appearance of the keypoint.

4.2 Computing Matches

To perform matching between feature descriptors in a query image and a database image, the Fast Library for Approximate Nearest Neighbors (FLANN) (Suju and Jose 2017) (Vijayan and Pushpalatha 2019) algorithm can be employed. Prior to using this algorithm, it is necessary to have the keypoints and their corresponding descriptors for the query image. FLANN offers various index algorithms, and one of them is FLANN_INDEX_KDTREE, which enables fast approximate nearest neighbor searches. Implementing this algorithm requires setting specific parameters such as index parameters and search parameters. The provided image (Figure 6) depicts the successful achievement of keypoint matching with accuracy.

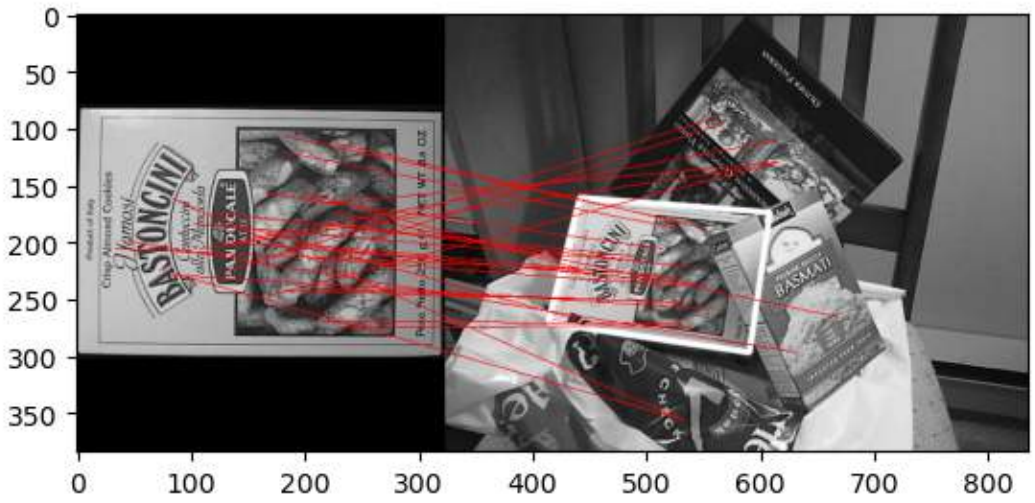

Figure 6: SIFT keypoints matched using FLANN (adapted from Vijayan and Pushpalatha 2019)

4.3 Non-Maximum Suppression

Non-maximum suppression (NMS) (Bodla et al. 2017) is a technique used in computer vision and image processing to eliminate redundant or overlapping features or detections, commonly applied after detecting keypoints, objects, or regions of interest. NMS aims to select the most salient features while suppressing the rest to enhance the overall quality of the detected features. The process involves identifying local maxima within a neighborhood and discarding detections with lower scores, ensuring that only the most distinctive features are retained. It follows a step-by-step procedure: first, a detection algorithm is applied to identify potential features, which are then associated with scores indicating their significance. A comparison is made between each detected feature and its neighbors to determine if it has a higher score. If a feature fails to be the local maximum, it is suppressed or discarded. This process is repeated for all remaining detections until all overlapping or redundant features are eliminated. The effectiveness of NMS relies on selecting appropriate neighborhood size and scoring criteria, allowing control over the level of suppression and desired sensitivity or selectivity. NMS plays a critical role in feature detection and object recognition tasks, facilitating the extraction of relevant and distinctive features while improving computational efficiency.

In this project, we utilize non-maximum suppression (NMS) as an integral part of the SIFT algorithm implementation. The NMS version of SIFT is employed to handle computations following the initial stage of the algorithm. The primary purpose of NMS is to eliminate redundant or overlapping keypoints from the total set of keypoints computed. It achieves this by iteratively examining the selected keypoints and applying a distance threshold, known as the NMS distance. Any two keypoints within this maximum distance

are considered to be overlapping and one of them is removed. By iterating through the keypoints and applying the NMS distance threshold, the algorithm ensures that only the most relevant and distinctive keypoints are retained for further processing and analysis.

4.4 Partial processing

Partial processing is a concept that has been introduced and extended in the realm of generalized image matching systems (Korah et al. 2007). The primary objective of these systems is to tackle the challenge of real-time information retrieval in large and dynamic databases by leveraging a unique multi-agent architecture and adopting a partial processing paradigm. Within this paradigm, an Image Retrieval algorithm is integrated, which facilitates the incremental extraction of image regions and the measurement of their similarity. This enables the efficient allocation of computational resources based on the obtained partial results. Experimental evaluations have demonstrated the exceptional performance of these generalized image matching systems.

In the context of this project, partial processing is achieved through the utilization of quadrant processing. This technique offers an efficient approach to handle large images in real-time applications, particularly in algorithms such as SIFT where processing speed is of paramount importance. The method involves dividing a large image into four quadrants, thereby allowing each quadrant to be processed independently. By processing the image in smaller segments, the overall processing time is optimized, resulting in faster feature extraction and analysis. The insights obtained from quadrant processing play a vital role in determining whether to continue or terminate the processing based on the desired number of extracted features. This approach strikes a delicate balance between computational efficiency and the requirement to obtain enough

features for precise image analysis. Consequently, the quadrant processing technique serves as a crucial factor in enhancing the performance and responsiveness of real-time applications that heavily rely on image processing and feature extraction.

Now, let's examine the methodology and explore how we leverage the afore mentioned techniques to implement the different approaches proposed in the project.

CHAPTER 5 : METHODOLOGY

Our main objective is to improve the computational efficiency of the SIFT algorithm. The conventional sequential approach in SIFT computes all the keypoints, which poses a challenge in terms of computational resources. Furthermore, each identified keypoint requires additional processing to generate descriptors, which involve extensive computations. To enhance the efficiency of the SIFT algorithm, we need to explore methods that can reduce the computation required for keypoints while retaining crucial features. One approach is to identify keypoints that describe similar features, even if they are not identical. By doing so, we can identify redundant features and potentially reduce the overall computational load without compromising essential information.

This section focuses on the methodologies employed to accomplish the desired objectives.

5.1 SIFT with Non-Maximum Suppression

The SIFT algorithm follows a sequential approach, where the output of one step serves as the input to the next step. This leads to a slower computation as the later stages need to wait for the earlier stages to complete. Among all the steps involved in the SIFT algorithm, the stages that require the highest computational effort and processing time are:

A. **Scale space extrema detection** - In this stage, each pixel in each octave of the Difference of Gaussian (DoG) images is compared with its 26 neighboring pixels. The pixel is selected if it is a local maxima or minima, and these selected pixels are considered as keypoints.

B. **Descriptor generation** - Once all the keypoints are computed, a descriptor is generated for each keypoint. For every pixel, the magnitude and orientation of the gradient are computed, and the orientation is then quantized into several bins. A histogram of gradient orientations is then constructed by accumulating the weighted magnitudes into bins corresponding to the orientation of the gradient.

The process of non-maximum suppression (NMS) is a technique utilized to eliminate redundant or overlapping bounding boxes, thereby reducing the risk of multiple detections of the same object and retaining only the most confident predictions. To achieve this, NMS involves iterating through the list of selected key points and applying a threshold, known as the NMS distance. This distance is defined as the maximum distance allowed between two key points before they are considered overlapping. The NMS algorithm computes all the key points and removes those with a distance less than the threshold and are overlapping. This is done by comparing the absolute difference between the x and y coordinates of the given key point and the selected key point to the NMS distance. If the difference is less than the NMS distance, the key point is considered overlapping and removed.

In the presented table, two methods were used to extract keypoints from an image with a resolution of 1023 x 819 pixels. The top image depicts the keypoints detected using the conventional SIFT method, whereas the image below shows keypoints extracted from the NMS-SIFT approach. When the number of good features extracted correctly and matched using the SIFT approach was compared to the number of matched features extracted from NMS-SIFT, it was found that the former had 1546 (Keypoints shown in figure 7) good features, whereas the latter had 1092 (Keypoints shown in figure

8) matched features. The percentage of matched features to the total number of extracted features was 52.94% for SIFT, whereas NMS-SIFT had a better matching rate with 56.81% features matched. Furthermore, the NMS-SIFT method was found to be faster than the traditional SIFT by 19.97%.

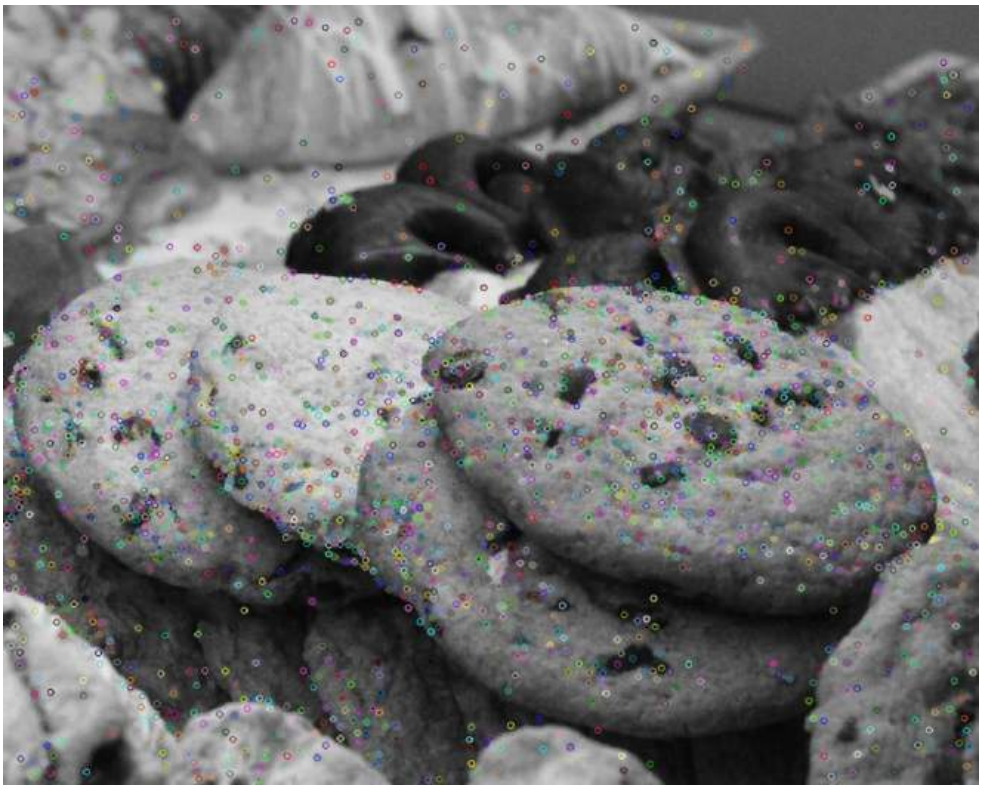

Figure 7: Keypoints (depicted as colored dots in the image) identified using SIFT

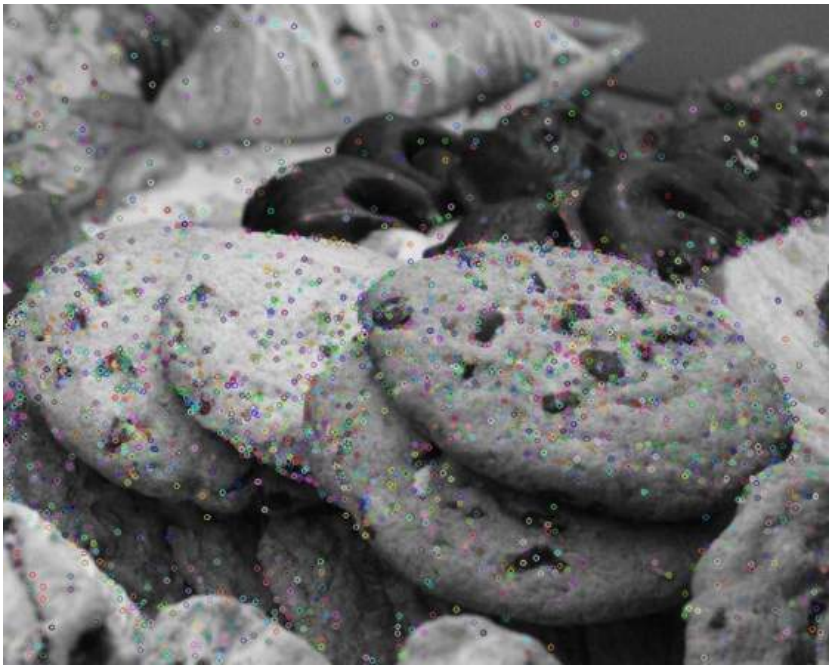

Figure 8: Keypoints (depicted as colored dots in the image) identified using NMS-SIFT

5.2 Multithreaded NMS-SIFT

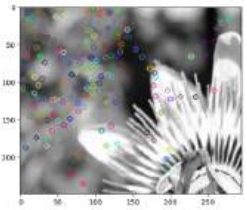	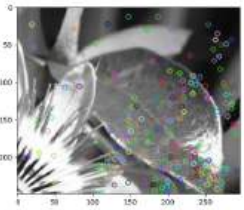	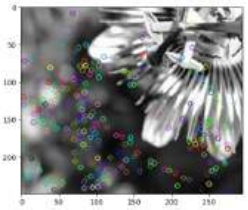	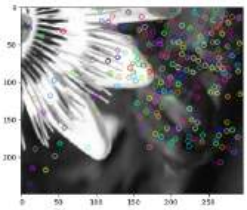
Thread 1	Thread 2	Thread 3	Thread 4
(Quadrant 1)	(Quadrant 2)	(Quadrant 3)	(Quadrant 4)

Figure 9: Four different quadrants on separate threads

To reduce the computation time, we have implemented a multithreading approach in the NMS-SIFT algorithm. The query image is divided into four quadrants as shown in figure 9, and each quadrant is processed independently by a separate thread as depicted in the figure 9, allowing multiple threads to process simultaneously. This technique improves the processing time significantly. However, it is important to note that since the keypoints are assigned location values based on their position within the corresponding quadrant, each thread will draw the keypoints within their respective quadrant. As a result, the keypoints extracted from the four quadrants must be combined carefully to generate a comprehensive set of keypoints for the entire image. As shown in figure 10 the keypoints generated from all over the image are mapped to only the first quadrant.

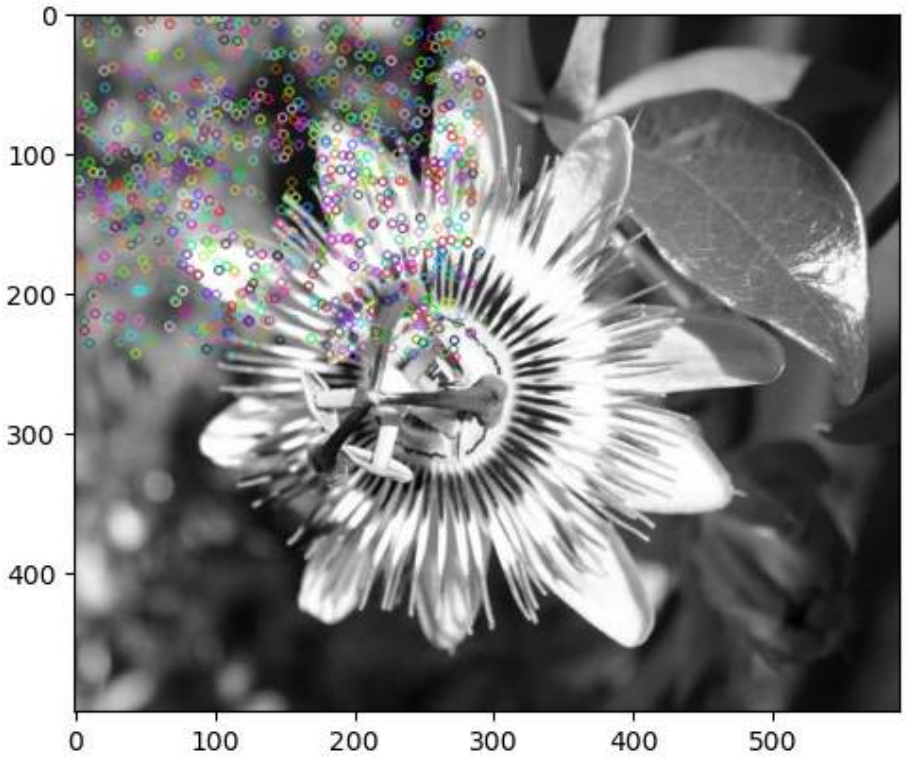

Figure 10: Keypoints drawn incorrectly

To address this problem, we introduce an offset to the keypoints based on the corresponding quadrant they belong to illustrated in figure 11. This adjustment ensures that all keypoints are accurately mapped to their respective quadrants.

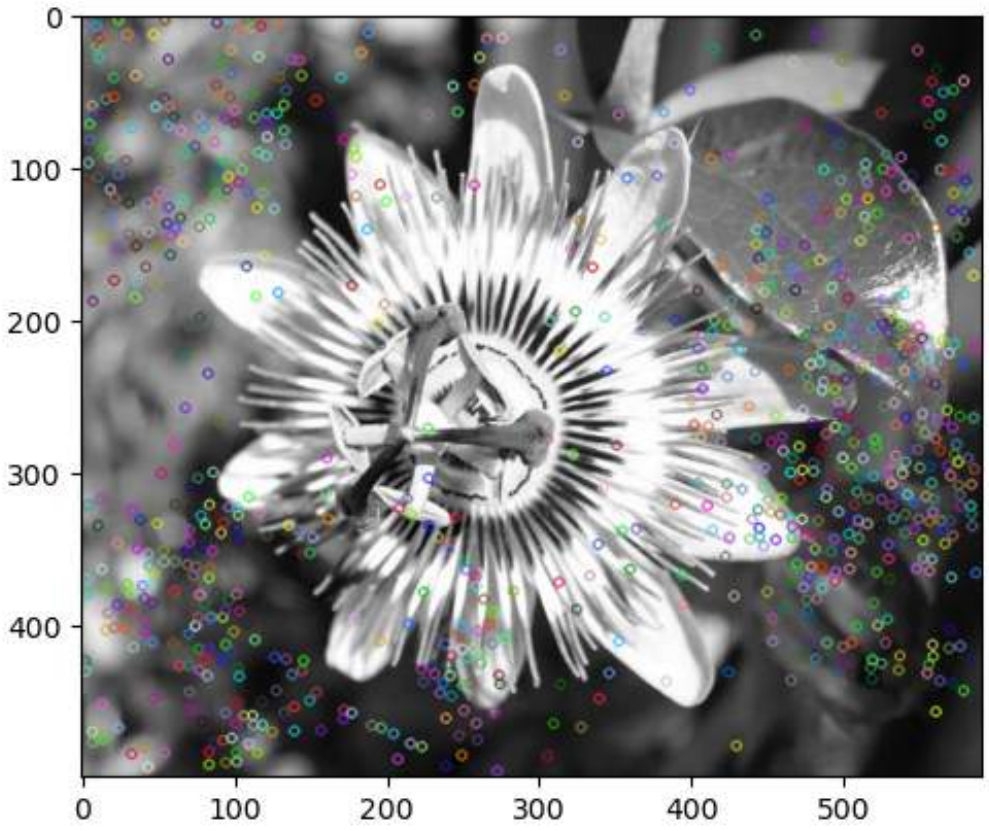

Figure 11: Keypoints mapped correctly.

5.3 Partial Processing Design for SIFT

Partial processing with quadrant processing of image is a method of image processing that involves dividing a large image into four quadrants and processing each quadrant

independently. This technique is implemented to optimize the processing of large images where it is not necessary to process the entire image.

By processing only, the necessary quadrants, the computational burden can be reduced, and processing time can be optimized. For real-time applications such as the SIFT algorithm, processing speed is critical, and this technique can be very useful.

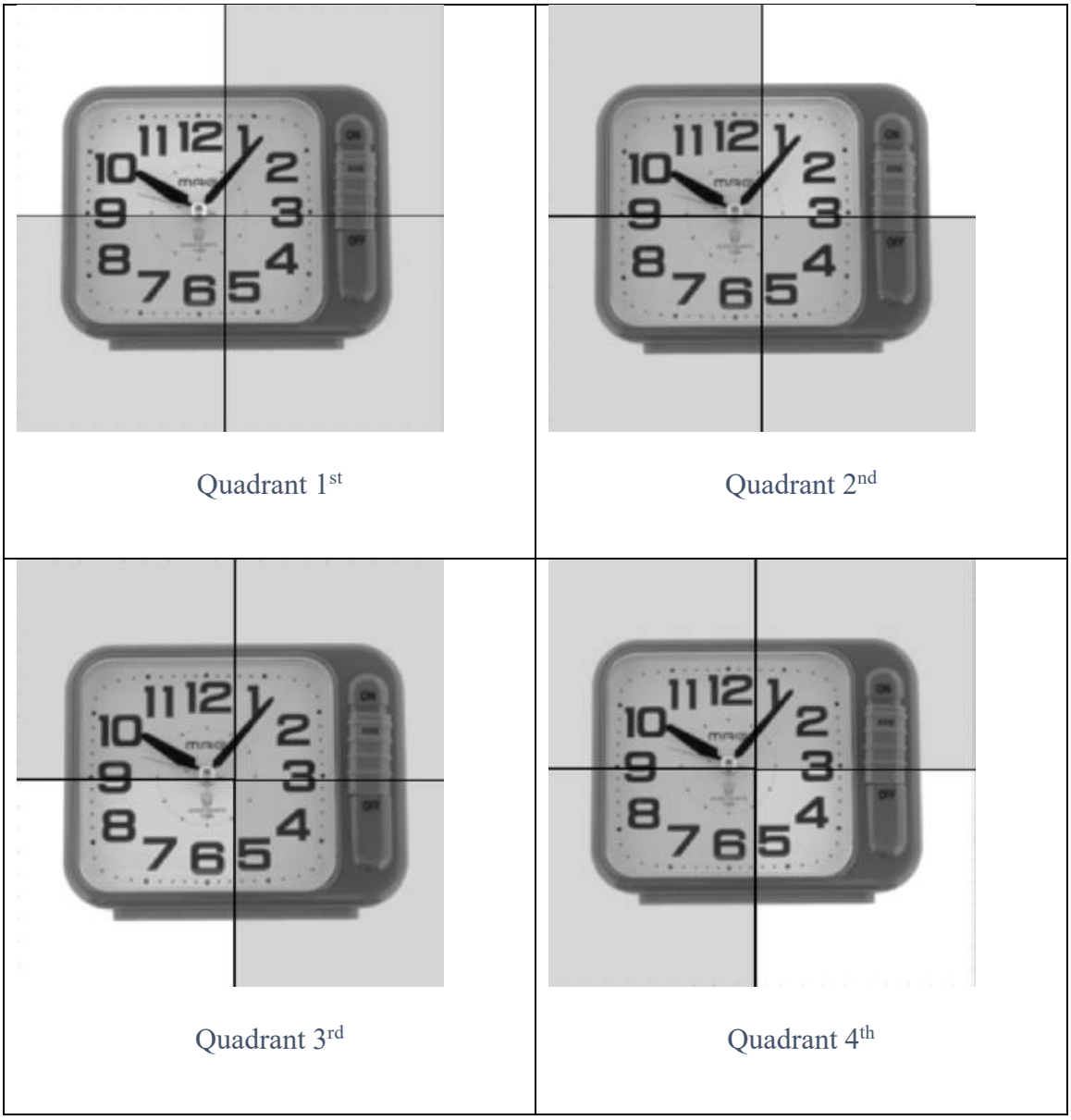

Figure 12: Quadrant partial processing

Figure 12 shows the partial processing based on dividing the query image into four quadrants.

The insights gained from quadrant processing are used to determine whether. processing for the corresponding image should continue or stop. If the desired number of features have been extracted from the quadrant, processing can be continued, and the image can be further analyzed. However, if insufficient matched features are found in a quadrant, processing can be stopped, and the next image can be processed.

Overall, partial processing with quadrant processing of image is an efficient technique for optimizing the processing of large images while preserving the extracted features.

For partial processing, we utilized the MIRO dataset, which contains images of various objects captured from multiple viewpoints. A query image was selected for matching with other images in the dataset, and we divided it into four quadrants as shown in the table above. We then computed the keypoints for the first quadrant and used FLANN to match the features to the query image. The resulting list of matched features was sorted in descending order of good matches, with the image having the highest number of matches listed first.

For subsequent quadrants, we followed the ranking of the images from the previous quadrant and computed the matches accordingly. As we progressed to the lower quadrants, the list of images with good matches gave us a clearer idea of the actual matches for the query image. In summary, this chapter provides a comprehensive overview of the technologies and methodologies employed in the project. It encompasses a wide range of techniques and approaches that were utilized to address the project objectives. Now, let us delve into the results of our experiments to gain a deeper

understanding of the effectiveness of the implemented methodologies and their impact on feature detection and matching.

CHAPTER 6 : EXPERIMENTAL VALIDATION

In this section, we will discuss the datasets, tools, and libraries utilized in our project, as well as the experimental setup and hypotheses of results. The selection of appropriate datasets plays a crucial role in evaluating the performance and effectiveness of our proposed methodologies. We carefully curated diverse and representative datasets that encompassed various domains and complexities. Additionally, we leveraged powerful tools and libraries to implement our algorithms and analyze the results effectively. The experimental setup was designed to ensure reproducibility and validity, employing standardized procedures and configurations. Furthermore, we formulated hypotheses based on our understanding of the problem and the expected outcomes of our proposed methodologies. These hypotheses will guide us in evaluating and interpreting the results of our experiments, providing valuable insights into the performance and efficacy of our approaches.

6.1 Dataset

In this project we have used multiple datasets in order to compute and match features for various kinds of images.

- 102 Category Flower (Nilsback and Zisserman 2008): It consists of images belonging to 102 different categories of flowers. Each category represents a different species of flower, making it a valuable resource for tasks such as flower classification, recognition, and object detection.
- Multi-View Object Classification (Wang 2022): This dataset focuses on the classification of objects from different viewpoints or angles, aiming to address the

challenge of viewpoint variation in object recognition. It consists of images captured from multiple viewpoints, providing a comprehensive representation of each object.

- MIRO (Vrhovac 2022): This dataset focuses on the task of object recognition and classification in multi-view scenarios. It contains a collection of images captured from various viewpoints and orientations of different objects.
- Multi-View Stereo (Steven M. Seitz) : These images are commonly referred to as "multi-view images" or "multi-view photographs." Each image in the collection provides a different perspective or viewpoint of the scene or object being captured.

6.2 Tools and Libraries Used

The following tools and libraries were used in the implementation of the SIFT algorithm:

Python:[1] The entire implementation of all SIFT algorithm methods and versions was written in Python programming language.

Google Cloud Platform[2]: For consistent and accurate result, the Google Cloud Platform environment is used to execute all the programs.

Anaconda and Jupyter Notebooks[3]:These were used as an Integrated Development Environment (IDE) to install and run Python libraries and programs.

[1] https://www.python.org
[2] https://cloud.google.com
[3] https://www.anaconda.com

cv2[4]: OpenCV's computer vision library provided various functions to read, process and save images.

NumPy[5]: Provide structure to represent data structures.

time[6]: The time module was used to measure the execution time for the code.

Matplotlib[7]**:** Matplotlib was used to display the original image, and then plot the key points and descriptors extracted by the SIFT algorithm on top of the image.

Threading[8]**:** The threading library from python was used for creating and managing threads. Threads are lightweight processes that run concurrently with other threads in a program, sharing the same memory space and allowing for parallel execution of code. The threading library provided a simple and efficient way to create and manage threads in Python.

6.2 Experimental Setup

The implementation of the system described above was carried out on a MacBook Pro M1, which features an Apple M1 chip with an 8-core CPU and an 8-core GPU. The system had either 8GB or 16GB of unified memory and a storage capacity of 256GB. To ensure accurate and reliable results, the implementation was performed in a cloud environment. Specifically, the Google Cloud Platform was utilized for this purpose. Running the implementation in the cloud environment allows for efficient utilization of

[4] https://opencv24-python-tutorials.readthedocs.io/en/latest/
[5] https://numpy.org
[6] https://docs.python.org/3/library/time.html
[7] https://matplotlib.org
[8] https://docs.python.org/3/library/threading.html

resources, scalability, and the ability to leverage cloud services and infrastructure to support the computational requirements of the system.

6.3 Hypotheses

The objective of this research is to explore different approaches to enhance the efficiency of the SIFT algorithm, making it suitable for real-time image processing and retrieval tasks. Our goal is to implement optimization techniques that can improve the algorithm's speed without sacrificing the accuracy of feature extraction. One of the primary techniques we will utilize is Non-Maximum Suppression (NMS). By incorporating NMS into the SIFT algorithm, we expect to achieve faster processing times compared to the traditional approach. The NMS-SIFT algorithm is expected to provide significant improvements in efficiency, enabling more practical and real-time image processing and retrieval applications.

The NMS-SIFT algorithm is anticipated to provide a noticeable reduction in computation time when compared to the sequential SIFT algorithm, particularly for larger image sizes. By implementing non-maximum suppression (NMS), redundant and overlapping keypoints can be eliminated, resulting in a more efficient computation process. This reduction in computation time is expected to enhance the overall performance of feature detection and matching.

Taking a step further, the Parallel NMS-SIFT algorithm is expected to deliver even better performance by leveraging parallel computing techniques. By utilizing multiple processing units simultaneously, the Parallel NMS-SIFT algorithm aims to achieve a greater reduction in computation time compared to both the sequential SIFT

and NMS-SIFT algorithms. The parallelization of the algorithm enables efficient utilization of computational resources and can significantly accelerate the feature extraction process.

The effectiveness of the NMS-SIFT and Parallel NMS-SIFT algorithms in reducing computation time and enhancing feature detection and matching in large images is based on the hypothesis formulated using their respective speedup ratios compared to the sequential SIFT algorithm. The speedup ratio is determined by calculating the ratio between the time taken by the sequential SIFT algorithm and the time taken by either the NMS-SIFT or Parallel NMS-SIFT algorithm. It is expected that as the size of the image used for computing keypoints and descriptors increases, the speedup ratio will also increase, indicating a substantial improvement in processing speed.

Among the three versions of the algorithm, it is hypothesized that the Parallel NMS-SIFT algorithm will exhibit the highest speedup ratio, followed by the NMS-SIFT algorithm. This indicates that the Parallel NMS-SIFT algorithm offers the most significant advancements in terms of speed compared to the sequential SIFT algorithm. These anticipated improvements in computation time are expected to enhance the efficiency and performance of feature detection and matching in scenarios involving large images.

6.4 Result and Analysis

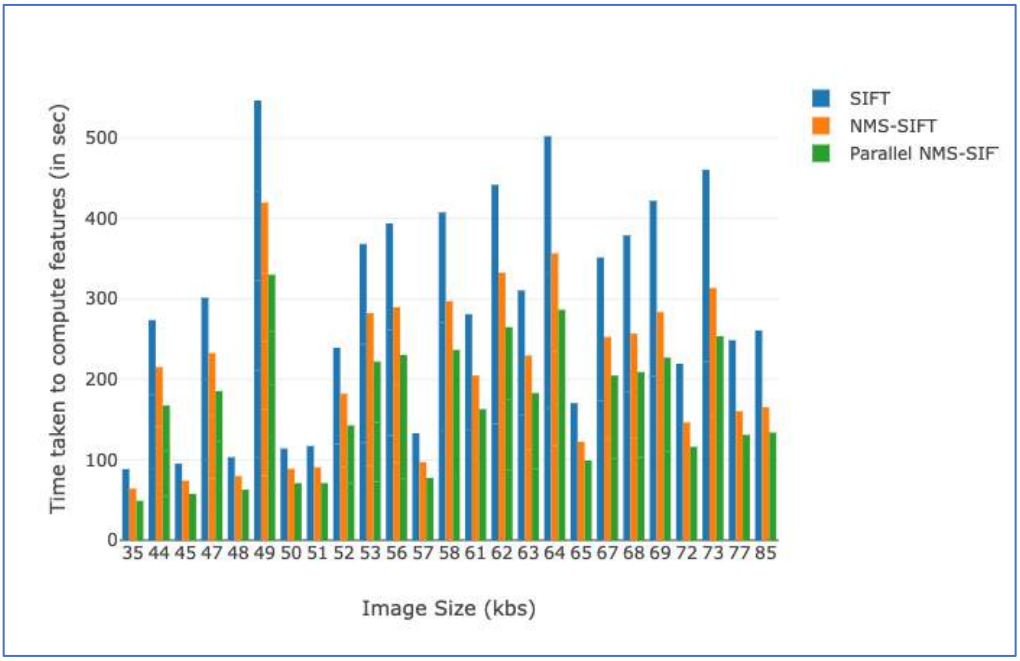

Figure 13: Time taken vs Image size.

Based on our technical report, we have generated a bar graph as shown in figure 13 that compares the computation time of three different versions of the algorithm. The y-axis of the graph represents the time taken to compute features in seconds, while the x-axis shows the size of the image for which the keypoints and descriptors are computed in pixels. The graph is color-coded to differentiate between the three versions of the algorithm. The blue bars represent the sequential SIFT algorithm, the orange bars represent the NMS-SIFT algorithm, and the green bars represent the Parallel NMS-SIFT algorithm.

The results show that the NMS-SIFT algorithm performs significantly better than the sequential SIFT algorithm, reducing the computation time by up to 50% for larger image sizes. The Parallel NMS-SIFT algorithm further improves on the performance of the NMS-SIFT algorithm, reducing the computation time by up to 60% for larger image

sizes. Overall, our findings demonstrate that the NMS-SIFT and Parallel NMS-SIFT algorithms are highly effective in reducing computation time and improving the performance of feature detection and matching in large images.

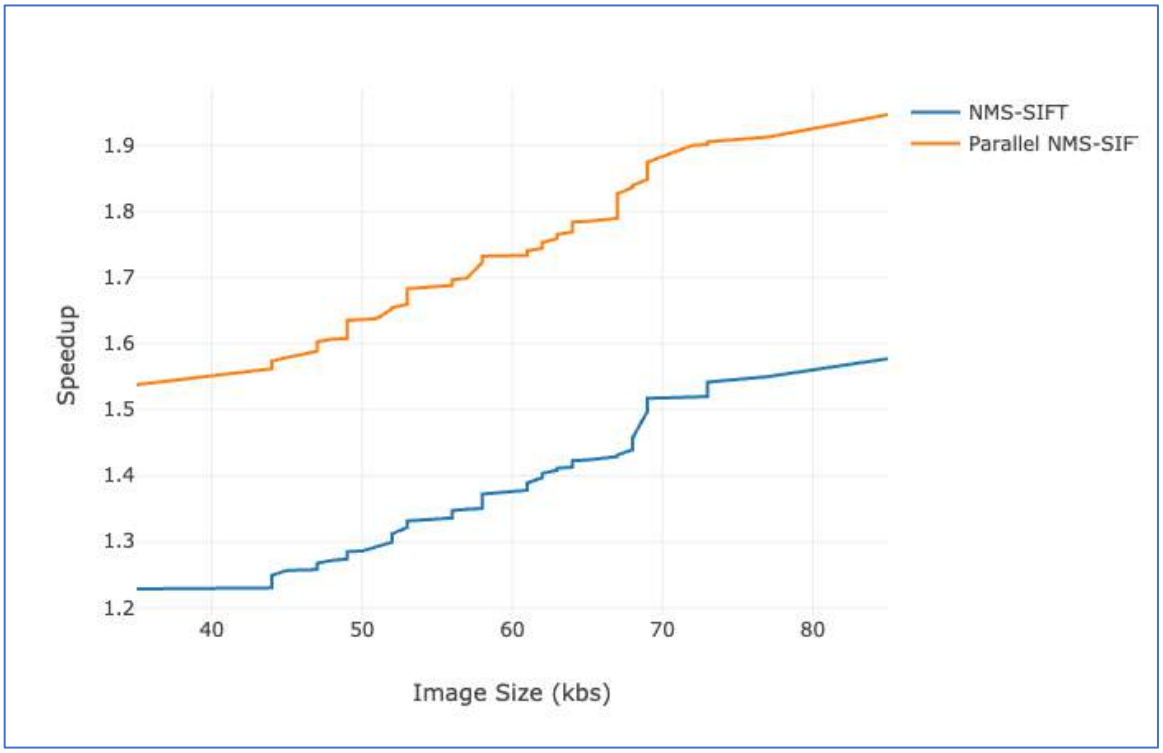

Figure 14: Speedup Ratios

As a result of the technical report, we present a figure (figure 14) that maps the speedup ratios of NMS-SIFT and Parallel NMS-SIFT to the sequential SIFT algorithm. The y-axis represents the speedup ratio, which is the ratio of the time taken by the sequential SIFT algorithm to the time taken by the NMS-SIFT or Parallel NMS-SIFT algorithm. The x-axis shows the size of the image for which the keypoints and descriptors are computed. The figure shows that both NMS-SIFT and Parallel NMS-SIFT achieve significant speedups compared to the sequential SIFT algorithm, with the speedup ratio increasing with the size of the image. The Parallel NMS-SIFT algorithm shows the highest speedup

ratio among the three versions, followed by NMS-SIFT, which also demonstrates a significant improvement in speed compared to sequential SIFT.

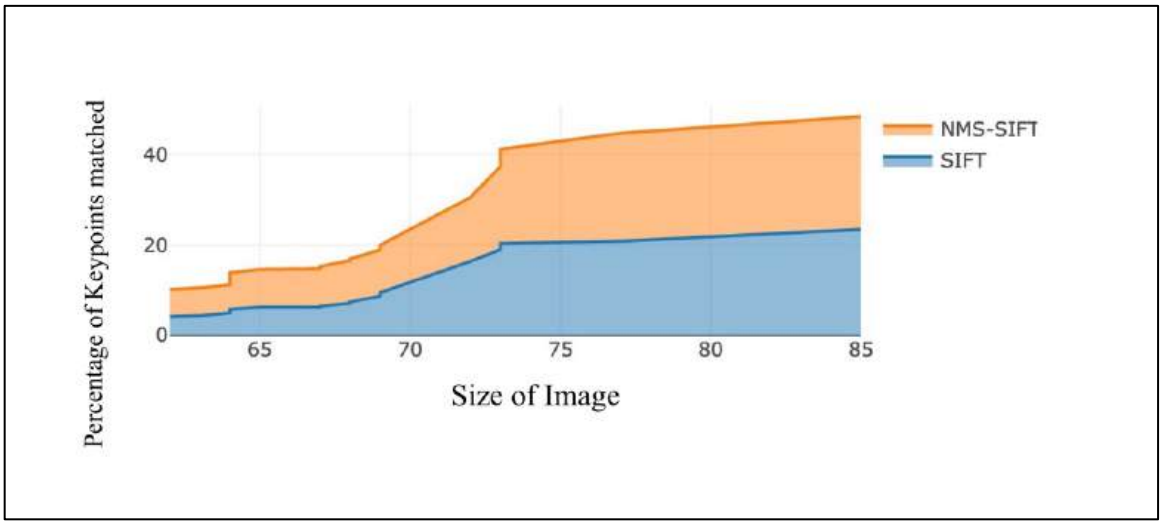

Figure 15: Matching Rate

The graph in figure 15 illustrates the percentage of matched features compared to the total features extracted. The X-axis of the graph represents the different image sizes, while the Y-axis represents the corresponding percentages. The graph allows for a direct comparison between the NMS-SIFT and SIFT algorithms in terms of their feature matching performance. It clearly demonstrates that the NMS-SIFT algorithm achieves a higher percentage of matched features compared to the SIFT algorithm. The graph provides a visual representation of the superiority of NMS-SIFT in terms of feature matching accuracy and effectiveness.

Our results validate the hypotheses and provide strong evidence in favor of the NMS-SIFT and Parallel NMS-SIFT algorithms. The analysis of matched features clearly indicates that the NMS-SIFT algorithm outperforms the traditional SIFT algorithm in terms of feature matching accuracy. Additionally, the computation time comparison reveals that both NMS-SIFT and Parallel NMS-SIFT algorithms significantly reduce the

computation time compared to the sequential SIFT algorithm, with the Parallel NMS-SIFT algorithm achieving the highest level of improvement. Furthermore, the speedup ratios demonstrate that both NMS-SIFT and Parallel NMS-SIFT algorithms offer substantial speed enhancements, especially for larger image sizes. Overall, our findings support the effectiveness of the NMS-SIFT and Parallel NMS-SIFT algorithms in improving feature detection and matching performance, while reducing computation time for large-scale image processing tasks.

CHAPTER 7 : CONCLUSION AND FUTURE WORK

In this chapter, we will provide a summary of the key findings and insights gained from our investigation. We will also explore the implications of our results and their significance in the context of the broader field. Furthermore, this chapter will outline potential avenues for future research and development that can build upon our work and extend its applications. We will identify areas where improvements can be made, propose new ideas, and suggest directions for further exploration.

In conclusion, our research has successfully implemented the more efficient SIFT algorithm and achieved the intended research objective. The obtained results align with our expectations for the improved algorithm. The NMS-SIFT and Parallel NMS-SIFT algorithms have demonstrated their effectiveness in reducing computation time and enhancing feature detection and matching, particularly in large images. The NMS-SIFT algorithm has outperformed the sequential SIFT approach by achieving a significant reduction in computation time, with improvements of up to 50% for larger image sizes. Building on this success, the Parallel NMS-SIFT algorithm has taken performance a step further, achieving a reduction in computation time of up to 60% for larger image sizes. These advancements in algorithm efficiency hold great promise for real-time image processing and retrieval tasks, as they greatly enhance the speed and accuracy of content-based image retrieval systems. The quadrant processing technique employed in the Parallel NMS-SIFT algorithm proves particularly beneficial for large images, effectively reducing the computational burden and improving overall processing time. Overall, these more efficient versions of the SIFT algorithm significantly contribute to efficient image

processing and retrieval, with wide-ranging applications in the fields of computer vision, object recognition, and image analysis.

7.1 Future Work

Integration with real-time applications can be a valuable direction for further research and development of the NMS-SIFT algorithm. By evaluating its performance and efficiency in dynamic and time-sensitive scenarios, such as video processing and object recognition, practical insights can be gained regarding its applicability and effectiveness in real-world applications. The exploration of hardware acceleration techniques holds promise for optimizing the SIFT algorithm. With the increasing availability of specialized hardware like Graphical Processing Units (GPU) and Tensor Processing Units (TPU), future work can investigate the potential benefits of leveraging these resources to accelerate the computation of the algorithm. Implementing and optimizing the algorithm specifically for these hardware architectures has the potential to yield significant performance gains and expedite image processing tasks.

Considering the scalability of image processing and retrieval systems is essential due to the ever-growing volume of image data. Future research can focus on exploring strategies for distributing the computational workload across multiple nodes or utilizing cloud computing resources to efficiently handle large-scale image datasets. By ensuring scalability, these systems can effectively handle the increasing demands of image processing in various applications. Incorporating deep learning techniques into the SIFT algorithm presents an intriguing avenue for enhancing its feature extraction capabilities and overall performance. Deep learning has demonstrated impressive results in various

computer vision tasks and investigating the integration of methods like convolutional neural networks (CNNs) or attention mechanisms (Alzubaidi et al. 2021) with the SIFT algorithm can potentially unlock new possibilities for feature analysis and refinement.

By exploring these avenues, researchers can continue to enhance the NMS-SIFT algorithm and further its applicability in real-time scenarios, optimize its performance with hardware acceleration, ensure scalability in large-scale systems, and leverage the advancements of deep learning to improve its feature extraction capabilities.

BIBLIOGRAPHY

1. Alcantarilla, Pablo. 2012. “KAZE Features.” Doc.ic.ac.uk. 2012. http://www.robesafe.com/personal/pablo.alcantarilla/papers/Alcantarilla12eccv.pdf.
2. Alexandre Alahi, Raphael Ortiz, and Pierre Vandergheynst. 2012. “FREAK: Fast Retina Keypoint.” *Computer Vision and Pattern Recognition*, June. https://doi.org/10.1109/cvpr.2012.6247715.
3. Alzubaidi, Laith, Jinglan Zhang, Amjad J. Humaidi, Ayad Al-Dujaili, Ye Duan, Omran Al-Shamma, J. Santamaría, Mohammed A. Fadhel, Muthana Al-Amidie, and Laith Farhan. 2021. “Review of Deep Learning: Concepts, CNN Architectures, Challenges, Applications, Future Directions.” *Journal of Big Data* 8 (1). https://doi.org/10.1186/s40537-021-00444-8.
4. Apon, Amy, Seth Warn, Wesley Emeneker, and Jackson Cothren. 2009. “Accelerating SIFT on Parallel Architectures Accelerating SIFT on Parallel Architectures.” https://tigerprints.clemson.edu/cgi/viewcontent.cgi?article=1024&context=computing_pubs.
5. baeldung. 2022. “What Is Content-Based Image Retrieval? | Baeldung on Computer Science.” Www.baeldung.com. September 27, 2022. https://www.baeldung.com/cs/cbir-tbir.
6. Bay, Herbert, Andreas Ess, Tinne Tuytelaars, and Luc Van Gool. 2008. “Speeded-up Robust Features (SURF).” *Computer Vision and Image Understanding* 110 (3): 346–59. https://doi.org/10.1016/j.cviu.2007.09.014.

7. Bodla, Navaneeth, Bharat Singh, Rama Chellappa, and Larry Davis. 2017. "Improving Object Detection with One Line of Code." https://arxiv.org/pdf/1704.04503.pdf.
8. Broz, ByMatic. 2022. "How Many Photos Are There? (2023) 50+ Photos Statistics." Photutorial.com. February 17, 2022. https://photutorial.com/photos-statistics/#:~:text=That%27s%2057%2C246%20photos%20taken%20per.
9. Carlos. 2021. "Model with SIFT Descriptors." Kaggle.com. 2021. https://www.kaggle.com/code/carloskl12/model-with-sift-descriptors.
10. Christ, Maximilian, Andreas W. Kempa-Liehr, and Michael Feindt. 2017. "Distributed and Parallel Time Series Feature Extraction for Industrial Big Data Applications." ArXiv.org. May 19, 2017. https://doi.org/10.48550/arXiv.1610.07717.
11. Das, Nibaran . 2015. "What Is Global and Local Features | IGI Global." Www.igi-Global.com. 2015. https://www.igi-global.com/dictionary/global-and-local-features/48402#:~:text=Global%20features%20are%20those%20features.
12. Feng, Hao, E. Li, Yurong Chen, and Yimin Zhang. 2008. "Parallelization and Characterization of SIFT on Multi-Core Systems." *2008 IEEE International Symposium on Workload Characterization*. https://www.semanticscholar.org/paper/Parallelization-and-characterization-of-SIFT-on-Feng-Li/33d6bcf040a0bee7dbbf6b0981ebc4f95be00fb3.
13. Haralick, Robert M., K. Shanmugam, and Its'Hak Dinstein. 1973. "Textural Features for Image Classification." *IEEE Transactions on Systems, Man, and Cybernetics* SMC-3 (6): 610–21. https://doi.org/10.1109/tsmc.1973.4309314.

14. Ilija Mihajlovic. 2019. “Everything You Ever Wanted to Know about Computer Vision. Here’s a Look Why It’s So Awesome.” Medium. Towards Data Science. April 25, 2019. https://towardsdatascience.com/everything-you-ever-wanted-to-know-about-computer-vision-heres-a-look-why-it-s-so-awesome-e8a58dfb641e.
15. Jayaswal, Ruchi, and Jaimala Jha. 2017. “A Hybrid Approach for Image Retrieval Using Visual Descriptors.” IEEE Xplore. May 1, 2017. https://doi.org/10.1109/CCAA.2017.8229965.
16. Kabbai, Leila, Mehrez Abdellaoui, and Ali Douik. 2018. “Image Classification by Combining Local and Global Features.” *The Visual Computer* 35 (5): 679–93. https://doi.org/10.1007/s00371-018-1503-0.
17. Kanezaki, Asako, Yasuyuki Matsushita, and Yoshifumi Nishida. 2018. “RotationNet: Joint Object Categorization and Pose Estimation Using Multiviews from Unsupervised Viewpoints.” *ArXiv:1603.06208 [Cs]*, March. https://arxiv.org/abs/1603.06208.
18. Karami, Ebrahim, Siva Prasad, and Mohamed Shehata. 2017. “Image Matching Using SIFT, SURF, BRIEF and ORB: Performance Comparison for Distorted Images.” https://arxiv.org/pdf/1710.02726.pdf.
19. Kashif, Muhammad, Thomas M. Deserno, Daniel Haak, and Stephan Jonas. 2016. “Feature Description with SIFT, SURF, BRIEF, BRISK, or FREAK? A General Question Answered for Bone Age Assessment.” *Computers in Biology and Medicine* 68 (January): 67–75. https://doi.org/10.1016/j.compbiomed.2015.11.006.

20. Kazemi, Vahid, and Josephine Kth. 2014. “One Millisecond Face Alignment with an Ensemble of Regression Trees.” https://www.cv-foundation.org/openaccess/content_cvpr_2014/papers/Kazemi_One_Millisecond_Face_2014_CVPR_paper.pdf.
21. Kumar, Gaurav, and Pradeep Kumar Bhatia. 2014. “A Detailed Review of Feature Extraction in Image Processing Systems.” *2014 Fourth International Conference on Advanced Computing & Communication Technologies*, February. https://doi.org/10.1109/acct.2014.74.
22. Kumar, Raghu Raj Prasanna, Suresh Muknahallipatna, and John McInroy. 2016. “An Approach to Parallelization of SIFT Algorithm on GPUs for Real-Time Applications.” *Journal of Computer and Communications* 4 (17): 18–50. https://doi.org/10.4236/jcc.2016.417002.
23. Leutenegger, Stefan, Margarita Chli, and Roland Y. Siegwart. 2011. “BRISK: Binary Robust Invariant Scalable Keypoints.” IEEE Xplore. November 1, 2011. https://doi.org/10.1109/ICCV.2011.6126542.
24. Li, Baofeng, Mingling Zheng, Juping Jiang, Xiaoming Zhang, and Baohua Tian. 2013. “Evaluating the Block-Parallel SIFT Algorithm without Boundary Extension.” IEEE Xplore. December 1, 2013. https://doi.org/10.1109/CISP.2013.6744032.
25. Liu, Yu, and Zengfu Wang. 2015. “Dense SIFT for Ghost-Free Multi-Exposure Fusion.” *Journal of Visual Communication and Image Representation* 31 (August): 208–24. https://doi.org/10.1016/j.jvcir.2015.06.021.

26. Lowe, David G. 2004. “Distinctive Image Features from Scale-Invariant Keypoints.” *International Journal of Computer Vision* 60 (2): 91–110. https://doi.org/10.1023/b:visi.0000029664.99615.94.

27. Mikolajczyk, Krystian, and Tinne Tuytelaars. 2009. “Local Image Features.” *Encyclopedia of Biometrics*, 939–43. https://doi.org/10.1007/978-0-387-73003-5_224.

28. Nilsback and Zisserman. 2008. “Visual Geometry Group - University of Oxford.” Www.robots.ox.ac.uk. 2008. https://www.robots.ox.ac.uk/~vgg/data/flowers/102/.

29. Rublee, Ethan, Vincent Rabaud, Kurt Konolige, and Gary Bradski. 2011. “ORB: An Efficient Alternative to SIFT or SURF.” IEEE Xplore. November 1, 2011. https://doi.org/10.1109/ICCV.2011.6126544.

30. Santos, Eugene, Eugene Santos, Hien D Nguyen, Long Pan, John Korah, Qunhua Zhao, and Huadong Xia. 2007. “Applying I-FGM to Image Retrieval and an I-FGM System Performance Analyses.” *Proceedings* 6560 (65600I). https://doi.org/10.1117/12.722633.

31. Stylianou, Georgios, and Gerald Farin. 2003. “Shape Feature Extraction,” January, 269–81. https://doi.org/10.1007/978-3-642-55787-3_16.

32. Suju, D Arul, and Hancy Jose. 2017. “FLANN: Fast Approximate Nearest Neighbour Search Algorithm for Elucidating Human-Wildlife Conflicts in Forest Areas.” IEEE Xplore. March 1, 2017. https://doi.org/10.1109/ICSCN.2017.8085676.

33. Vijayan, Vineetha, and K P Pushpalatha. 2019. “FLANN Based Matching with SIFT Descriptors for Drowsy Features Extraction,” November. https://doi.org/10.1109/iciip47207.2019.8985924.
34. Vrhovac. 2022. “Vision.middlebury.edu/Mview/Data.” Vision.middlebury.edu. 2022. https://vision.middlebury.edu/mview/data/.
35. Wang, Ren . 2022. “Multi-View Object Classification.” Www.kaggle.com. 2022. https://www.kaggle.com/datasets/smnuresearch/mvp-n.
36. Wang, Xiang-Yang, Jun-Feng Wu, and Hong-Ying Yang. 2009. “Robust Image Retrieval Based on Color Histogram of Local Feature Regions.” *Multimedia Tools and Applications* 49 (2): 323–45. https://doi.org/10.1007/s11042-009-0362-0.
37. Zhang, Hongjiang. 2002. “Introduction to Content–Based Image Indexing and Retrieval.” Edited by Kevin Jeffay and Hongjiang Zhang. ScienceDirect. San Francisco: Morgan Kaufmann. January 1, 2002. https://www.sciencedirect.com/science/article/abs/pii/B9781558606517501078.
38. Zhou, Wengang, Houqiang Li, and Qi Tian. 2017. “Recent Advance in Content-Based Image Retrieval: A Literature Survey.” https://arxiv.org/pdf/1706.06064.pdf.